

ЛАБОРАТОРНЫЕ РАБОТЫ

Методические указания предназначены для выполнения лабораторных работ по учебному программному эмулятору x86. Работы расположены по возрастанию сложности: от знакомства с интерфейсом до форматированного ввода-вывода и передачи параметров через стек.

Перед выполнением работ студент должен уметь запускать эмулятор, открывать окно ассемблера, компилировать программу, выполнять ее пошагово и просматривать регистры, флаги, память и консоль.

Общие требования к отчету

1. указать номер и тему лабораторной работы
2. привести цель работы
3. вставить или переписать текст программы
4. заполнить таблицы наблюдений
5. сформулировать вывод по результатам выполнения
6. ответить на контрольные вопросы

ЛАБОРАТОРНАЯ РАБОТА 1. Знакомство с интерфейсом эмулятора

Цель работы: Освоить запуск программы, основные окна эмулятора, пошаговое выполнение и наблюдение за регистрами.

Продолжительность лабораторной работы: 2 академических часа.
Самостоятельная подготовка: 1-2 часа.

1. Самостоятельная подготовка

1. Изучить назначение окон ассемблера, процессора, памяти и консоли.
2. Повторить назначение регистров EAX, EBX, ECX, EDX, ESP, EBP, EIP.
3. Подготовить короткую программу с инструкцией `int3`.

2. Пример программы

Листинг 1 - Исходная программа для лабораторной работы 1

main:

mov eax, 10h

mov ebx, 20h

int3

3. Порядок выполнения работы

1. Ввести программу в окне ассемблера.
2. Скомпилировать программу и убедиться, что ошибок нет.
3. Выполнить первую инструкцию в пошаговом режиме.
4. Записать значения EAX, EBX и EIP после каждого шага.
5. Запустить программу до int3 и проверить состояние остановки.

4. Форма для заполнения

Таблица 1 - Наблюдение за регистрами при пошаговом выполнении

Шаг	Выполненная команда	EAX	EBX	EIP	Состояние
1					
2					
3					

5. Содержание отчета

1. цель работы;
2. текст программы;
3. таблицу изменения регистров по шагам;
4. вывод о назначении режима пошагового выполнения.

6. Контрольные вопросы

1. Для чего используется EIP?
2. Чем пошаговое выполнение отличается от запуска программы?
3. Почему int3 удобно применять в учебных примерах?

ЛАБОРАТОРНАЯ РАБОТА 2. Арифметические команды и флаги

Цель работы: Изучить команды `add`, `sub`, `inc`, `dec`, `neg`, `cmp` и влияние результата на флаги.

Продолжительность лабораторной работы: 2 академических часа.
Самостоятельная подготовка: 1-2 часа.

1. Самостоятельная подготовка

1. Повторить назначение флагов `ZF`, `SF`, `CF`, `OF`.
2. Подготовить два набора исходных чисел: положительные и отрицательные.
3. Изучить отличие команды `cmp` от `sub`.

2. Пример программы

Листинг 2 - Исходная программа для лабораторной работы 2

`main:`

```
mov eax, 5  
mov ebx, 5  
sub eax, ebx  
cmp eax, 0  
inc eax  
neg eax  
int3
```

3. Порядок выполнения работы

1. Выполнить программу пошагово.
2. После каждой арифметической команды записать `EAX` и флаги.
3. Изменить исходные значения так, чтобы результат `sub` был отрицательным.
4. Повторить запуск и сравнить состояние `SF` и `ZF`.
5. Объяснить, почему `cmp` меняет флаги, но не меняет операнды.

4. Форма для заполнения

Таблица 2 - Изменение флагов после арифметических команд

Команда	EAX	ZF	SF	CF	OF
sub eax, ebx					
cmp eax, 0					
inc eax					
neg eax					

5. Содержание отчета

1. текст программы;
2. таблицу значений EAX, ZF, SF, CF, OF;
3. вывод о связи результата арифметики и флагов.

6. Контрольные вопросы

1. Когда устанавливается ZF?
2. Чем CF отличается от OF?
3. Почему команда cmp нужна перед условными переходами?

ЛАБОРАТОРНАЯ РАБОТА 3. Память, строки и директивы db/dd

Цель работы: Освоить размещение данных в памяти, работу с метками, строками UTF-8 и 32-битными значениями.

Продолжительность лабораторной работы: 2 академических часа.
Самостоятельная подготовка: 1-2 часа.

1. Самостоятельная подготовка

1. Изучить директивы db и dd.
2. Повторить назначение offset.
3. Повторить, почему русские символы в UTF-8 занимают несколько байтов.

2. Пример программы

Листинг 3 - Исходная программа для лабораторной работы 3

```
start:
    jmp main

msg db "Привет!", 0
value dd 123

main:
    mov edx, offset msg
    mov ebx, offset value
    int3
```

3. Порядок выполнения работы

1. Скомпилировать программу.
2. Найти в памяти адрес метки msg.
3. Записать байты строки и определить завершающий нулевой байт.
4. Найти адрес value и проверить размещение 32-битного значения.
5. Объяснить, чем db отличается от dd.

4. Форма для заполнения

Таблица 3 - Размещение данных в памяти

Метка	Адрес	Содержимое памяти	Комментарий
msg			
value			

5. Содержание отчета

1. текст программы;
2. адреса msg и value;
3. фрагмент памяти со строкой и числом;
4. вывод о представлении данных в памяти.

6. Контрольные вопросы

1. Для чего нужен offset?
2. Почему строка должна завершаться нулевым байтом?
3. Сколько байтов занимает dd?

ЛАБОРАТОРНАЯ РАБОТА 4. Условные переходы и циклы

Цель работы: Изучить работу `cmp`, условных переходов и простого цикла со счетчиком.

Продолжительность лабораторной работы: 2 академических часа.
Самостоятельная подготовка: 1-2 часа.

1. Самостоятельная подготовка

1. Повторить команды `jmp`, `je`, `jne`, `jl`, `jg`.
2. Изучить роль ZF, SF и OF в знаковых переходах.
3. Подготовить таблицу значений счетчика для цикла.

2. Пример программы

Листинг 4 - Исходная программа для лабораторной работы 4

`main:`

```
mov eax, 0  
mov ecx, 5
```

`loop_start:`

```
add eax, ecx  
dec ecx  
cmp ecx, 0  
jne loop_start  
int3
```

3. Порядок выполнения работы

1. Выполнить программу пошагово до первой команды `cmp`.
2. Записать значения EAX, ECX и ZF на каждой итерации.
3. Проверить, в какой момент переход `jne` перестает выполняться.
4. Изменить начальное значение ECX и повторить опыт.
5. Сделать вывод о связи `cmp`, ZF и условного перехода.

4. Форма для заполнения

Таблица 4 - Наблюдение за выполнением цикла

Итерац ия	EAX	ECX	ZF после cmp	Перехо д выполнен
1				
2				
3				
4				
5				

5. Содержание отчета

1. текст программы;
2. таблицу итераций цикла;
3. значение суммы после завершения;
4. ответы на контрольные вопросы.

6. Контрольные вопросы

1. Как команда jne определяет, нужно ли выполнять переход?
2. Почему перед условным переходом обычно стоит cmp?
3. Что произойдет, если убрать dec ecx?

ЛАБОРАТОРНАЯ РАБОТА 5. Стек, подпрограммы и параметры

Цель работы: Изучить команды push, pop, call, ret и изменение регистра ESP.

Продолжительность лабораторной работы: 2 академических часа.
Самостоятельная подготовка: 1-2 часа.

1. Самостоятельная подготовка

1. Повторить принцип работы стека.
2. Изучить назначение ESP и адреса возврата.
3. Подготовить схему изменения стека при call и ret.

2. Пример программы

Листинг 5 - Исходная программа для лабораторной работы 5

main:

```
mov eax, 7  
call double_value  
int3
```

double_value:

```
add eax, eax  
ret
```

3. Порядок выполнения работы

1. Записать начальное значение ESP.
2. Выполнить call и посмотреть, как изменился стек.
3. Определить адрес возврата в памяти.
4. Выполнить ret и проверить EIP.
5. Добавить push ebx и pop ebx вокруг вызова, повторить наблюдение.

4. Форма для заполнения

Таблица 5 - Изменение стека при call и ret

Момент выполнения	ESP	EIP	Содержимое вершины стека
До call			
После call			
Перед ret			
После ret			

5. Содержание отчета

1. текст программы;
2. значения ESP до call, после call и после ret;
3. объяснение, где хранится адрес возврата;
4. вывод о роли стека.

6. Контрольные вопросы

1. Почему call изменяет стек?
2. Что делает ret?
3. Почему после push обычно нужен pop или восстановление ESP?

ЛАБОРАТОРНАЯ РАБОТА 6. Консоль, syscall, printf и scanf

Цель работы: Освоить вывод строк, ввод данных, форматированный вывод и передачу параметров через стек.

Продолжительность лабораторной работы: 2 академических часа.
Самостоятельная подготовка: 1-2 часа.

1. Самостоятельная подготовка

1. Изучить номера syscall 1, 2, 3 и 4.
2. Повторить спецификаторы %d, %u, %x, %s.
3. Повторить правило восстановления ESP после передачи параметров.

2. Пример программы

Листинг 6 - Исходная программа для лабораторной работы 6

```
fmt db "value=%d", 0
```

```
main:
```

```
    mov eax, 42  
    push eax  
    mov eax, 3  
    mov edx, offset fmt  
    syscall  
    add esp, 4  
    int3
```

3. Порядок выполнения работы

1. Выполнить программу и проверить вывод в консоли.
2. Записать значение ESP до push, после push и после add esp, 4.
3. Изменить формат на шестнадцатеричный вывод.
4. Составить программу ввода строки через syscall read.
5. Проверить, что ввод учитывается только в состоянии ожидания.

4. Форма для заполнения

Таблица 6 - Проверка системных вызовов

Операция	EAX	EDX	ESP до	ESP после	Результат
printf					
read					
scanf					

5. Содержание отчета

1. текст программ для printf и read;
2. скриншот или описание вывода консоли;
3. таблицу изменения ESP;
4. вывод о передаче параметров через стек.

6. Контрольные вопросы

1. Почему параметры printf передаются через стек?
2. Зачем после syscall выполнять add esp, N?
3. Почему ввод строки заранее не должен учитываться программой?